

PRACTICAL SESSION 4

TASK 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
(base) saiabburi@Geyasree-MacBook-Air Practical4 % clang wait_demo.c -o wait_demo
(base) saiabburi@Geyasree-MacBook-Air Practical4 % ./wait_demo
Parent process started. PID = 4423
Child 1: PID = 4424

Parent using wait()...
Child 2: PID = 4425
Child 3: PID = 4426
Child 1 completed.
wait(): Child PID 4424 finished with status 10

Parent using waitpid() for Child 3...
Child 2 completed.
Child 3 completed.
waitpid(): Child 3 PID 4426 finished with status 30

Parent using waitpid() for Child 2...
waitpid(): Child 2 PID 4425 finished with status 20

[All child processes completed.]
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t child1, child2, child3;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    child1 = fork();

    if (child1 == 0) {
        printf("Child 1: PID = %d\n", getpid());
        sleep(1);
        printf("Child 1 completed.\n");
        exit(10);
    }

    child2 = fork();

    if (child2 == 0) {
        printf("Child 2: PID = %d\n", getpid());
        sleep(3);
        printf("Child 2 completed.\n");
        exit(20);
    }

    child3 = fork();

    if (child3 == 0) {
        printf("Child 3: PID = %d\n", getpid());
        sleep(5);
        printf("Child 3 completed.\n");
        exit(30);
    }

    printf("\nParent using wait()...\n");
```

```

pid_t finished = wait(&status);

if (WIFEXITED(status)) {
    printf("wait(): Child PID %d finished with status %d\n",
        finished, WEXITSTATUS(status));
}

printf("\nParent using waitpid() for Child 3...\n");

waitpid(child3, &status, 0);

if (WIFEXITED(status)) {
    printf("waitpid(): Child 3 PID %d finished with status %d\n",
        child3, WEXITSTATUS(status));
}

printf("\nParent using waitpid() for Child 2...\n");

waitpid(child2, &status, 0);

if (WIFEXITED(status)) {
    printf("waitpid(): Child 2 PID %d finished with status %d\n",
        child2, WEXITSTATUS(status));
}

printf("\nAll child processes completed.\n");

return 0;
}

```

TASK 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

(base) saiabburi@Geyasree-MacBook-Air Practical4 % nano zombie.c
(base) saiabburi@Geyasree-MacBook-Air Practical4 % clang zombie.c -o zombie
(base) saiabburi@Geyasree-MacBook-Air Practical4 % ./zombie
Parent process: PID = 4438
Child PID = 4439
Parent sleeping for 30 seconds without wait()...
Child process: PID = 4439
Child is exiting...
Parent finished.
(base) saiabburi@Geyasree-MacBook-Air Practical4 % nano zombie.c
(base) saiabburi@Geyasree-MacBook-Air Practical4 % clang zombie.c -o zombie
(base) saiabburi@Geyasree-MacBook-Air Practical4 % ./zombie
Parent process: PID = 4498
Child PID = 4499
Parent is waiting for the child...
Child process: PID = 4499
Child is exiting...
Child terminated normally.
Zombie process eliminated.
(base) saiabburi@Geyasree-MacBook-Air Practical4 % ps -axo pid,ppid,state,command | grep zombie
4501  4360 R+   grep zombie
(base) saiabburi@Geyasree-MacBook-Air Practical4 %

```

UW PICO 5.09

File: zombie.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child is exiting...\n");
        exit(0);
    }
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping for 30 seconds without wait()...\n");

        sleep(30);

        printf("Parent finished.\n");
    }

    return 0;
}

```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child is exiting...\n");
        exit(0);
    }
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent is waiting for the child...\n");

        wait(&status);

        if (WIFEXITED(status)) {
            printf("Child terminated normally.\n");
        }

        printf("Zombie process eliminated.\n");
    }

    return 0;
}
```